Enterprise React & GraphQL / REST Architektur

Vollständige Ordnerstruktur & Modulverantwortlichkeiten (Dokumentation)

Vollständiger Projekt-Verzeichnisbaum (src/)

```
src/
├── app/                          # Anwendungsweite Infrastruktur & Initialisierung
│   ├── providers/                # Alle React Context Provider (Apollo, Auth, Theme, Toast)
│   │   ├── ApolloProvider.jsx    # GraphQL Client Provider & InMemoryCache
│   │   ├── AuthProvider.jsx      # User-Session, Login-Zustand & Auth-Context
│   │   ├── ThemeProvider.jsx     # Dark/Light Mode & Styling-Theme
│   │   └── index.jsx             # AppProvider-Wrapper (verhindert Provider-Hölle)
│   └── routes/                   # Routing-Definitionen & Guard-Komponenten
│       ├── AppRoutes.jsx         # Zentrale Route-Table (React Router / TanStack Router)
│       └── ProtectedRoute.jsx    # Guard-Komponente für geschützte Seiten
│
├── shared/                       # Global wiederverwendbarer, domänenfreier Code
│   ├── api/                      # Zentrale HTTP / GraphQL Client-Konfiguration
│   │   ├── httpClient.js         # axios.create() mit BaseURL & Interceptors
│   │   └── apolloClient.js       # ApolloClient-Instanz mit Links & Error-Handling
│   ├── components/               # Generic Design System UI-Komponenten
│   │   ├── Button.jsx            # Generischer Button
│   │   ├── Modal.jsx             # Wiederverwendbarer Dialog
│   │   └── Input.jsx             # Formular-Eingabefeld
│   ├── graphql/                  # Plattformweite, universelle GraphQL-Definitionen
│   │   └── fragments/            # Echte globale Fragments (PageInfo, BaseUser, etc.)
│   │       ├── pagination.gql.js # PageInfo Fragment für alle paginierten Queries
│   │       └── user.gql.js       # Minimales User-Fragment für Avatare/Header
│   └── utils/                    # Helper-Funktionen ohne React-Abhängigkeit
│       ├── formatDate.js         # Datums-Formatierung
│       └── pdfExporter.js        # Generischer PDF/Blob-Exporter
│
├── features/                     # Fachliche Domänen / Feature-Module (Modular Design)
│   ├── projects/                 # Feature: Projektverwaltung
│   │   ├── api/                  # Queries, Mutationen & API-Aufrufe NUR für Projects
│   │   │   └── projects.queries.js
│   │   ├── components/           # UI-Komponenten spezifisch für Projects
│   │   │   ├── ProjectCard.jsx
│   │   │   └── ProjectCard.gql.js# Colocated Fragment NUR für ProjectCard
│   │   ├── hooks/                # Custom React Hooks für Projects (z. B. useProjects)
│   │   └── index.js              # Public API des Features (kapselt Interna)
│   │
│   ├── tasks/                    # Feature: Aufgabenverwaltung
│   │   ├── api/
│   │   │   └── tasks.queries.js
│   │   ├── components/
│   │   └── index.js
│   │
│   └── comments/                 # Feature: Kommentarfunktion
│
└── __generated__/                # Automatisch generierte TypeScript-Typen (GraphQL CodeGen)
```

Erklärung der Hauptbereiche

Ordner	Inhalt & Aufgabe
`src/app`	Gibt der gesamten Anwendung Struktur. Hält Context Provider (Auth, Apollo, Theme) und die Routing-Tabelle. Unabhängig von konkreten Fachdomänen.
`src/shared`	Verfasser von globalen Bausteinen. Enthält die Instanzen von HTTP-Clients (`axios.create()`), Design-System UI (Button, Modal), reine Utilities und universelle Fragments (`PageInfo`).
`src/features`	Kapselt fachliche Module (Projects, Tasks, Comments). Jedes Modul bringt seine eigenen UI-Komponenten, Lokale Fragments, API-Queries/Hooks und Logik isoliert mit.
`src/__generated__`	Vom *GraphQL Code Generator* automatisch erstellte TypeScript-Interfaces & Types für volle Typensicherheit.